



Branch of Study: Computer Science

Course: IT Project C1

Student Name: Otu-ekong Joseph Effiong

Student ID: 42468

Course Type: Full-time course

Institution Name: University of Economics and Human Sciences in Warsaw
Poland, Warsaw, Ul. Okopowa 59

Documentation for the diploma project prepared under the supervision of **Marcin Kacprowicz**

Academic Year: 2026

SkyLine: A Visa-Aware Low-Cost Flight Planning Web Application for Travelers with Low Passport Mobility

1. Basic Information

1.1. Project Name

SkyLine: a visa-aware low-cost flight planning web application for travelers with low passport mobility.

1.2. Project Goal

The goal of the project is to create a modern web application that helps foreigners and travelers with low passport strength find affordable, realistic, and legally safer flight routes. The application combines flight search, passport and visa eligibility checks, airport transit analysis, and route visualization to recommend travel plans that fit the user's nationality, existing visas, residence status, travel dates, and budget.

Unlike a standard flight search engine that only sorts results by price or duration, SkyLine evaluates whether a traveler is likely to be allowed to enter or transit through the countries included in the route. The system is designed to reduce the risk of choosing a cheap route that requires a transit visa, creates a self-transfer problem, or sends the traveler through an airport that is unsuitable for their passport situation.

1.3. Brief Description of the Project

SkyLine is a full-stack web application for visa-aware flight planning. A user creates a traveler profile containing passport country, nationality, residence country, passport expiry date, and existing visas or residence permits. The user then enters the origin, destination, preferred dates, and budget. The system searches for possible flight routes, checks destination and transit rules, scores each route, and presents the safest low-cost options with a clear explanation.

The application includes an airport search module, a route recommendation engine, a visa and transit rule engine, a flight path map, saved traveler profiles, saved searches, booking records, and payment simulation. Skyline is especially useful for users whose travel options are limited by visa requirements and passport mobility. The main output is not only a list of flights, but a complete recommended travel plan showing price, duration, layovers, transit countries, visa warnings, and the reason why a route is recommended or rejected.

2. Competitor Product Analysis

2.1. Google Flights

Google Flights is very effective for comparing prices, dates, airlines, and route duration. However, it does not deeply personalize recommendations based on the traveler's passport strength, current residence status, or existing visas. It may show cheap routes that are legally difficult or impossible for some travelers to use.

2.2. Skyscanner and Kiwi.com

Skyscanner and Kiwi.com are strong for low-cost route discovery and flexible date search. Kiwi.com is especially useful for creative routing and virtual interlining. However, these tools still focus mainly on price and availability. They do not act as a passport-aware travel planning assistant and may require the user to manually understand transit visa risk.

2.3. Sherpa and IATA Timatic

Sherpa and IATA Timatic focus on travel restrictions, visa documentation, and entry requirements. They are useful for official travel eligibility checks, but they are not complete flight planning interfaces. They do not usually optimize a route based on price, layover risk, and passport mobility together in one simple workflow.

2.4. FlightAware and OpenSky

FlightAware provides strong operational flight status information, while OpenSky provides aircraft position data. These tools are useful for tracking and visualization, but they are not booking systems and do not solve visa-aware route planning.

Compared to these products, SkyLine combines the main needs of a specific user group: low-cost flight planning, visa and transit awareness, passport-strength consideration, and route visualization. The application focuses on helping the user choose a route that is affordable and legally realistic, not simply the cheapest result.

3. List of Technologies Used

- React, Vite, JavaScript, Tailwind CSS, shadcn/ui components, and Lucide icons for the frontend.
- Netlify for hosting, HTTPS, deployment, environment variables, and serverless backend functions.
- Supabase for authentication, PostgreSQL database storage, saved traveler profiles, saved searches,

and booking records.

- Duffel API for flight search, pricing, passenger data, and order creation.
 - Kiwi.com Tequila API as an optional provider for low-cost and flexible route discovery.
 - FlightAware AeroAPI for flight status, delays, gates, terminals, and operational tracking.
 - OpenSky Network for free aircraft position data and map-based tracking.
 - OurAirports dataset for airport codes, airport search, city matching, and coordinates.
 - Custom visa and transit rule tables in Supabase for the diploma implementation.
 - IATA Timatic or Sherpa as possible future professional sources for travel-documentation rules.
 - Stripe test mode for payment simulation before booking confirmation.
-

4. Technological Stack and Justification

React and Vite were selected for the frontend because they provide a fast development environment and a modern structure suitable for a responsive travel planning interface. Tailwind CSS and shadcn/ui allow the interface to be built quickly with consistent components such as forms, route cards, alerts, dialogs, filters, and status badges.

Supabase was selected as the database and authentication platform because the Skyline requires relational data: users, traveler profiles, visas, passport information, saved searches, route recommenda-

tions, payments, and bookings. PostgreSQL is a strong fit because these records have clear relationships. Supabase Auth also reduces the amount of custom security code needed for user accounts.

I chose Duffel as the flight booking provider because it offers a modern REST API, sandbox environment, and a clear search, price, and book workflow. For cheaper and more flexible route discovery, Kiwi.com Tequila has been added as an optional search provider. FlightAware AeroAPI is suitable for operational flight status, while OpenSky is suitable for free aircraft position visualization. OurAirports is used locally to avoid unnecessary paid calls for airport lookup and to provide route coordinates for the flight path map.

I deployed the application using Netlify as it builds the frontend with `npm run build` and published from the `dist` directory. Netlify Functions can hold API keys safely and call external services without exposing secrets in the browser. This is important because keys for Duffel, FlightAware, Stripe, and Supabase service roles must remain private.

5. Key Issues Related to the Implementation of the Project

5.1. Visa-Aware Route Filtering

I had issues when ranking flights only by price, so I built a custom visa and transit rule engine that evaluates every flight route against the user's active profile passport profile before ranking. Each rule describes whether a traveler from one country can enter or transit through another country, whether an eVisa is available, whether visa on arrival is possible, and whether exceptions apply. In a production version, this data will be replaced or validated by professional services such as IATA Timatic or Sherpa. The engine considers passport country, destination country, transit countries, existing visas, residence permits, passport expiry, and whether airport transit is allowed. Rather than relying solely on third-party APIs for travel restrictions, I built dedicated relational tables in Supabase (`visa_rules`, `passport_strengths`, and `transit_rules`).

When a user searches for a route, the system checks:

5.1.1. Entry requirements for the destination based on the user's nationality and existing active visas or residence permits.

5.1.2. Transit requirements for all intermediate layover countries, verifying if airside transit is allowed without a visa or if a transit visa is required.

5.1.3. Passport validity constraints against the travel dates.

Routes that violate entry rules are categorized as "Avoid", while those with potential risks (such as required self-transfers or tight layover windows) are tagged with prominent warnings.

5.2. Route Scoring and Recommendation Logic

I've given Skyline the ability to rank each route using several factors. Visa safety has the highest weight because a legal or transit issue can make an otherwise cheap route unusable. Price is also important because the app targets users who need low-cost options. Duration, number of layovers, airport reliability, self-transfer risk, and baggage risk are also included.

A practical scoring model is:

- **Visa safety - 45%:** Deducts severe penalties for missing transit visas or dangerous self-transfers
- **Price - 30%:** Normalizes cost against the user's maximum budget and market average
- **Duration - 15%:** Penalizes long layovers and excessively indirect flights.
- **Comfort and layovers - 10%:** Evaluates risk factors such as multi-airport transfers or overnight

stays.

Routes with serious visa conflicts will be rejected or marked as “avoid”, even if they are cheaper. Routes with mild uncertainty will display warnings such as “transit visa may be required”, “check airline baggage transfer”, or “passport expiry risk”.

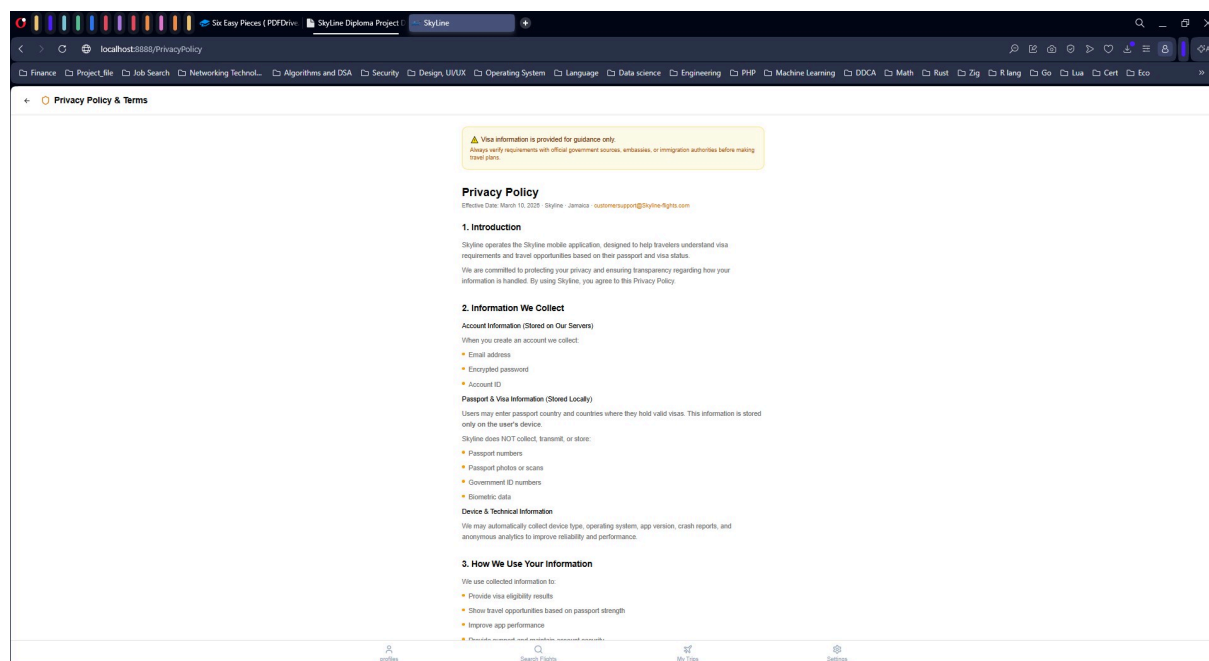
5.3. Secure Serverless API Integration

Flight booking APIs and payment APIs require private keys. These keys must never be stored in Vite frontend variables because browser code can be inspected by users. Skyline therefore uses a backend proxy through Netlify Functions or a Node.js/Express server. The frontend calls endpoints such as `/api/flights/search`, `/api/flights/price`, `/api/flights/book`, `/api/visa/check`, and `/api/payments/create-intent`. The backend then calls Duffel, FlightAware, Stripe, and Supabase securely.

5.4. Airport Search and Flight Path Visualization

The application needs reliable airport data for search and maps. I ingested the OurAirports dataset directly into Supabase with airport names, IATA codes, ICAO codes, city, country, latitude, and longitude. When a route is displayed, the app can draw the path from the origin airport to each layover and then to the destination. Each stop can show visa or transit warnings directly on the map or route card. It was all done to allow the user to lookup airports offline with map visualizations. An interactive leaflet mapping componenets in the frontend, rendering the full multi-leg flight path with color-coded stopovers representing transit risk levels.

5.5. Data Privacy and Sensitive Traveler Information



Skyline stores sensitive profile information such as passport country, nationality, visas, and travel history. I implemented strict Row Level Security (RLS) policies within Supabase, that minimizes and masks sensitive stored data and avoid saving unnecessary full passport details unless required. Allowing the user to access their own profiles, saved searches, and bookings. Card payments are processed via stripe elements, ensuring that no details are stored or touches our applications database or backend functions.

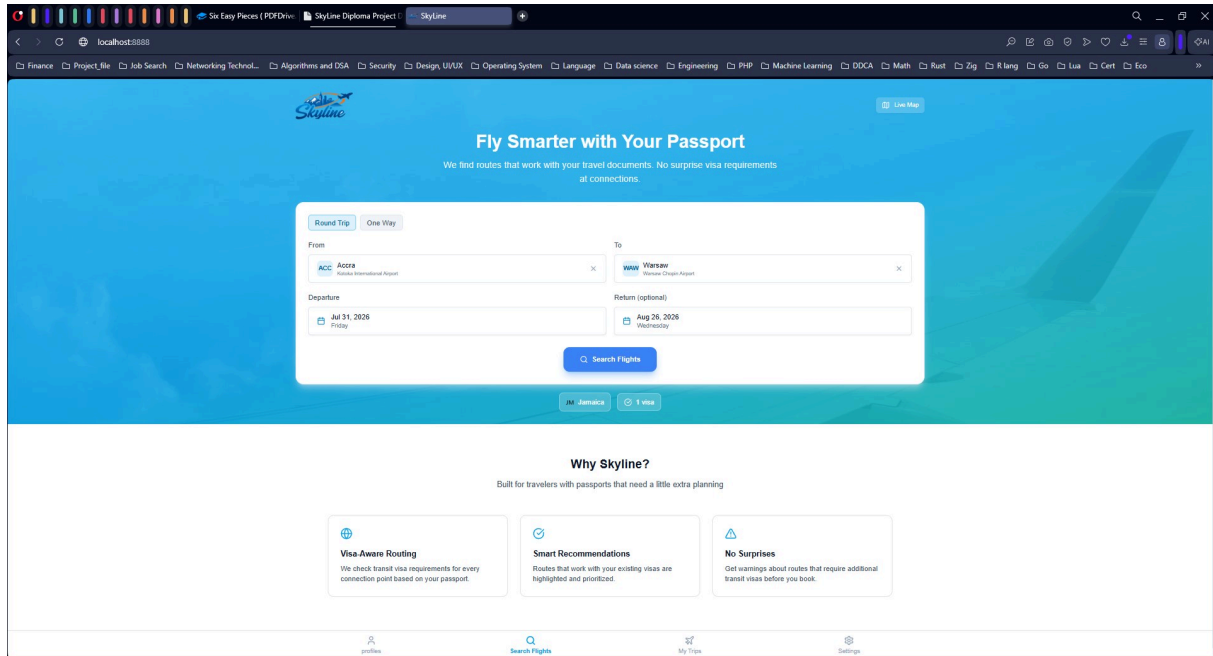
5.6. Handling Incomplete or Uncertain Travel Rules

Visa rules change frequently and can depend on many details, so I built the rule engine to fail safely. When a route involves an unmapped country pair or ambiguous transit policy, the system explicitly marks the status as “Verification Required” and provides actionable guidance on what the traveler should verify with official embassy portals before booking. This improves trust and makes the recommendation logic transparent.

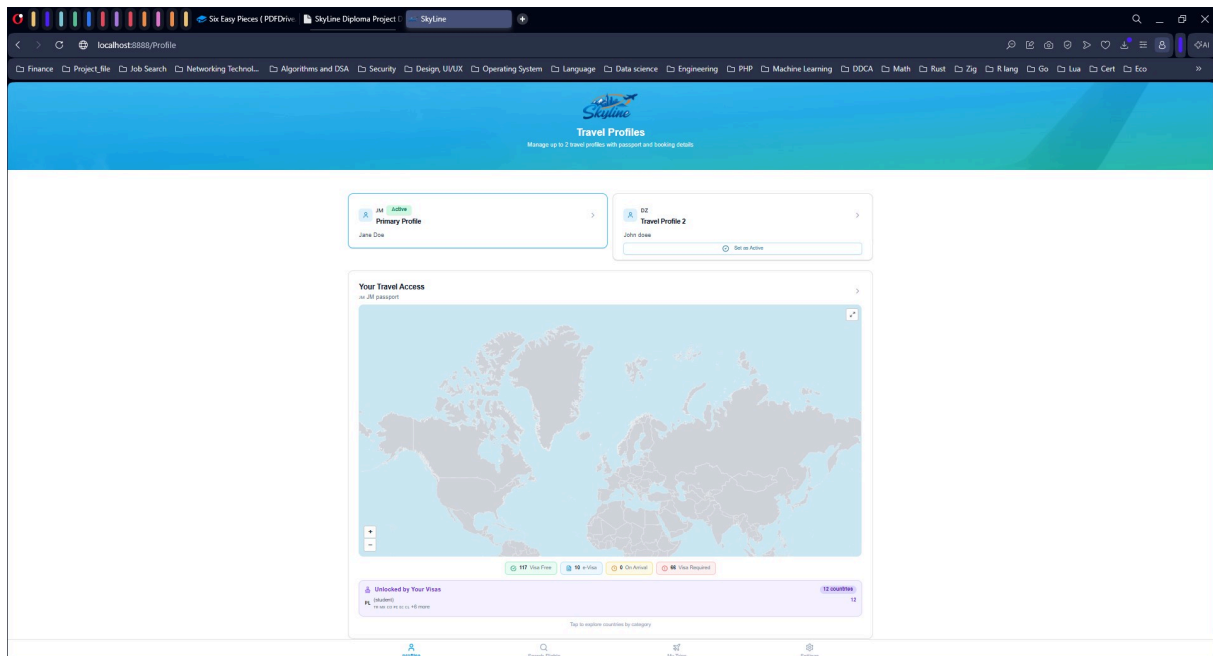
6. Suggested Screenshots and Visualizations

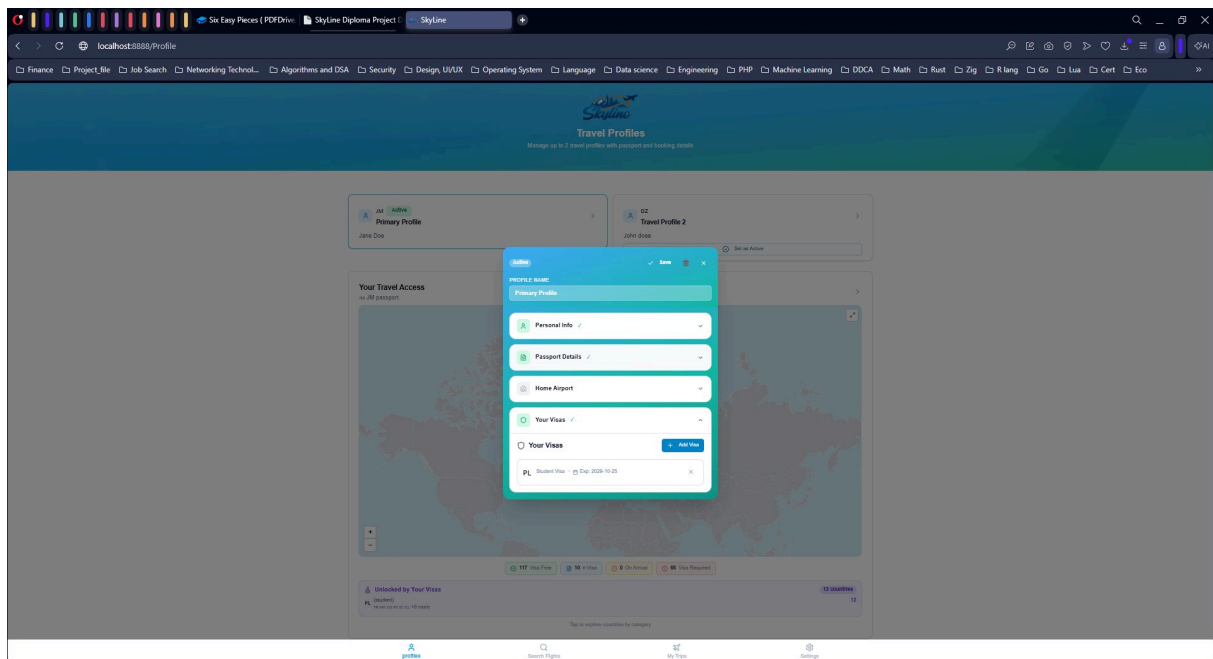
Screenshots that demonstrate the userflow of using the application:

- Home/search screen with origin, destination, dates

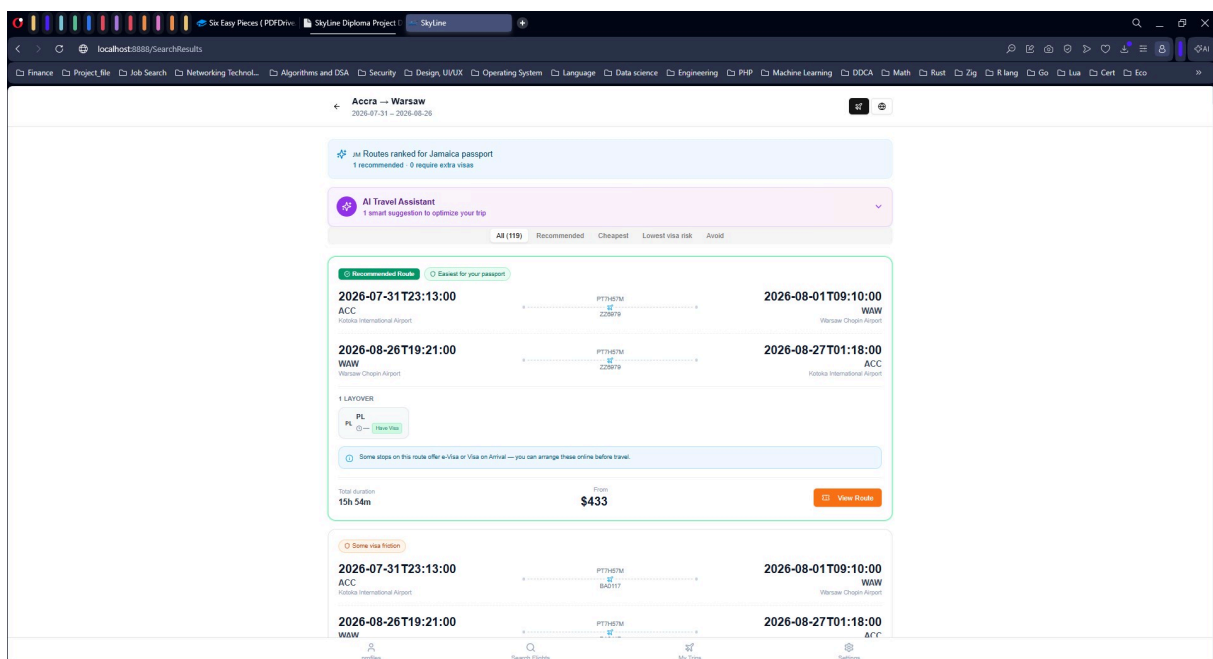


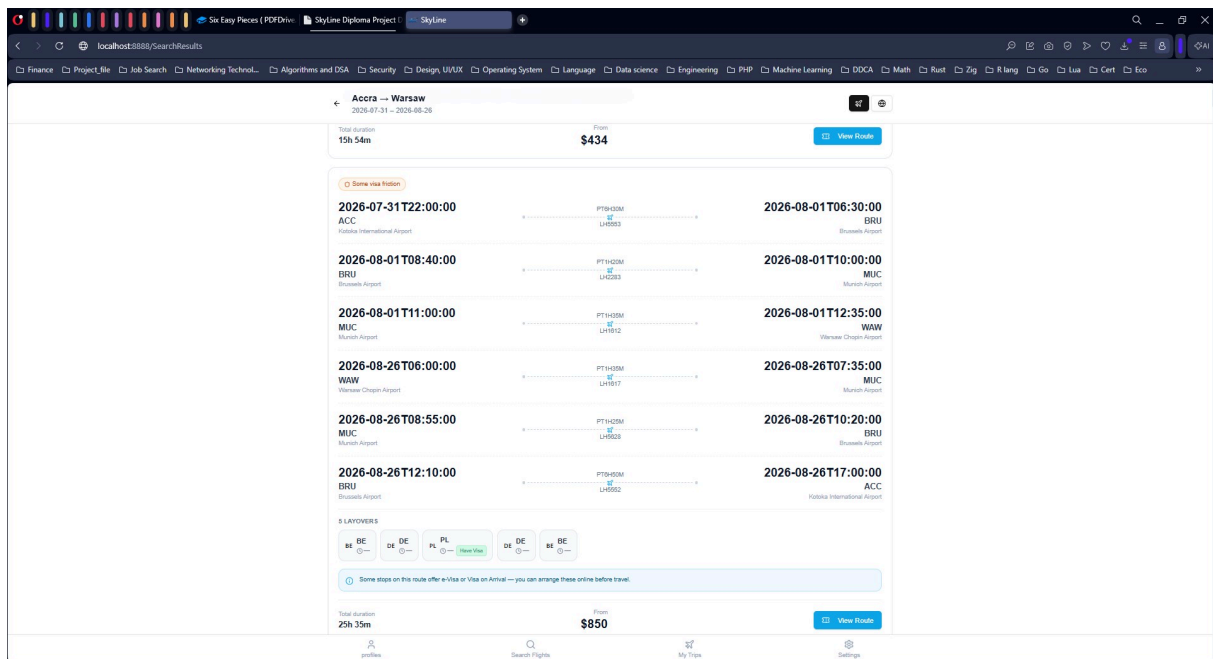
- Traveler profile screen showing saved passport country, residence country, and visas



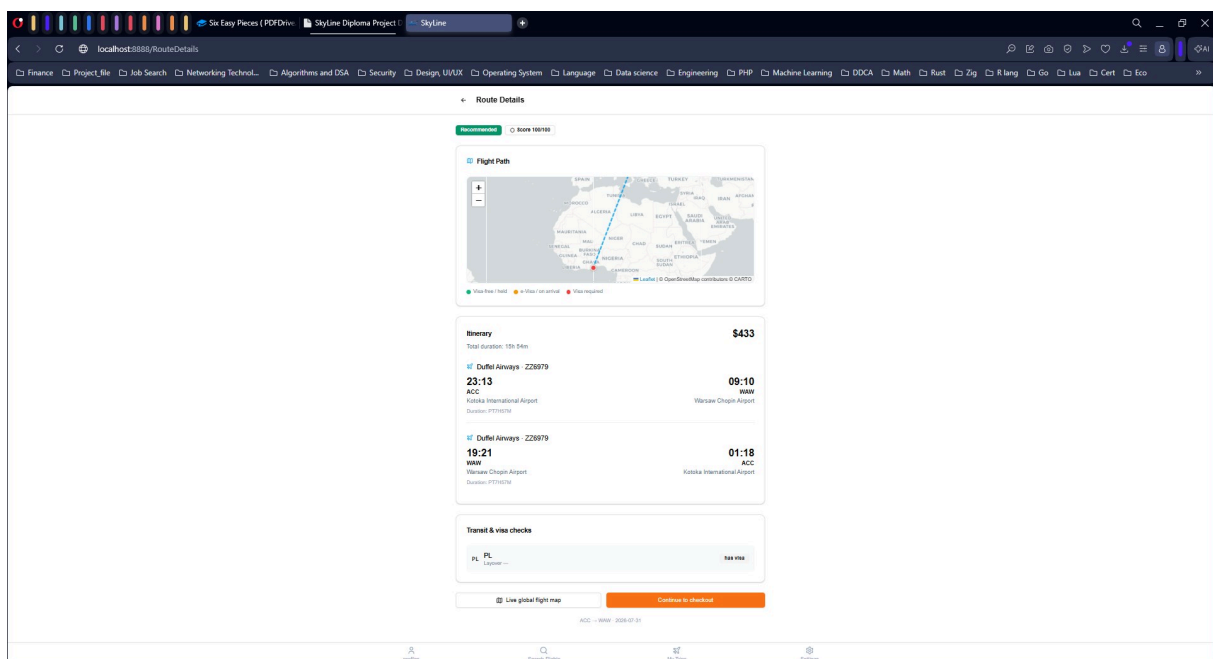


- Route results screen showing recommended, warning, and avoid route cards.





- Flight path map showing origin, layovers, destination.



- Booking/checkout screen using Stripe test mode and saving the booking to Supabase.

localhost:5888/Checkout

Finance Project_file Job Search Networking Technol... Algorithms and DSA Security Design, UI/UX Operating System Language Data science Engineering PHP Machine Learning DDCA Math Rust Zig R lang Go Lua Cert Eco

Checkout

Doha → Warsaw \$300

Passenger Details [A field from your profile](#)

Title Gender

Mr. Male

First name Last name

John doe

Date of birth

02/02/2000

Confirmation email

otajoseph345@yahoo.com

Phone number

+97463297261

(Digits after the country code - e.g. +974000000000 (no spaces or dashes))

Continue to Payment

localhost:5888/Checkout

Finance Project_file Job Search Networking Technol... Algorithms and DSA Security Design, UI/UX Operating System Language Data science Engineering PHP Machine Learning DDCA Math Rust Zig R lang Go Lua Cert Eco

Checkout

Doha → Warsaw \$300

Payment

Link [otajoseph345@yahoo.com](#)

Visa Credit

Use this card

Add a card

Card number

4242 4242 4242 4242

Expiration

03 / 39

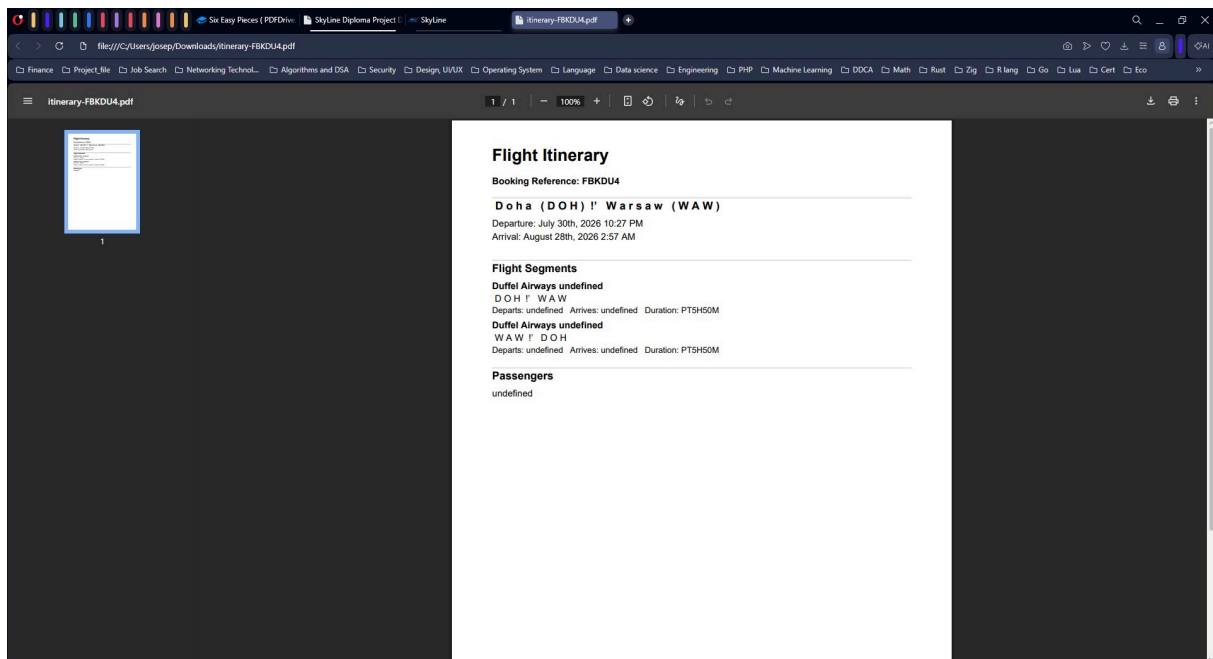
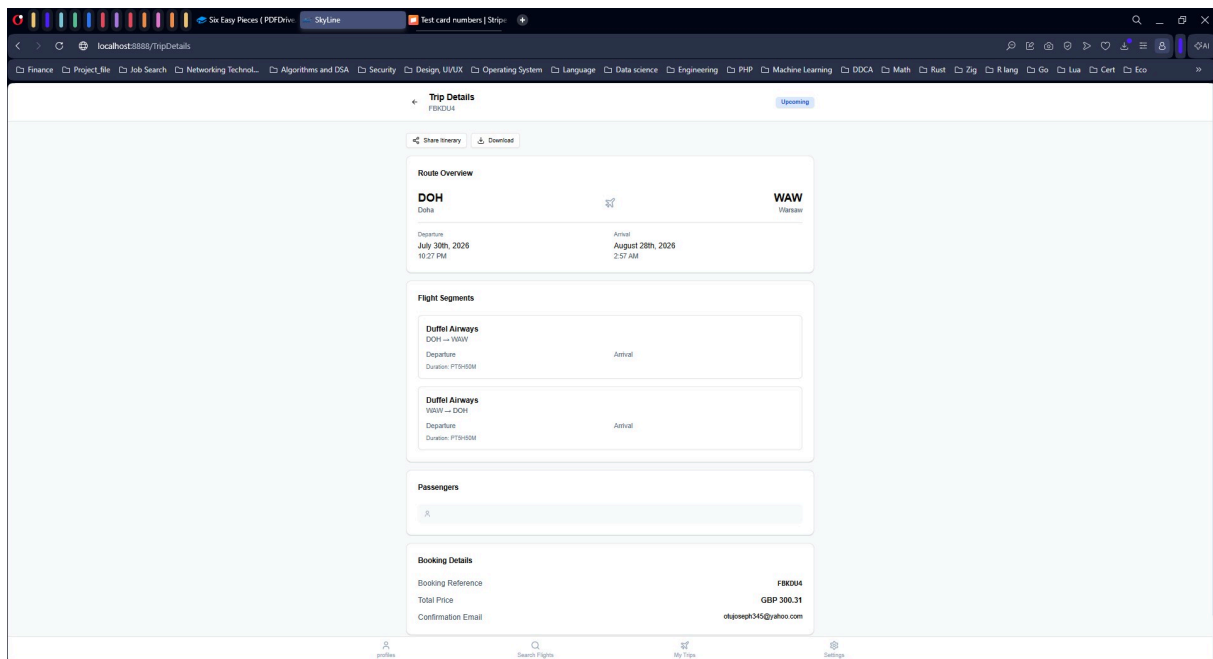
CVC

123

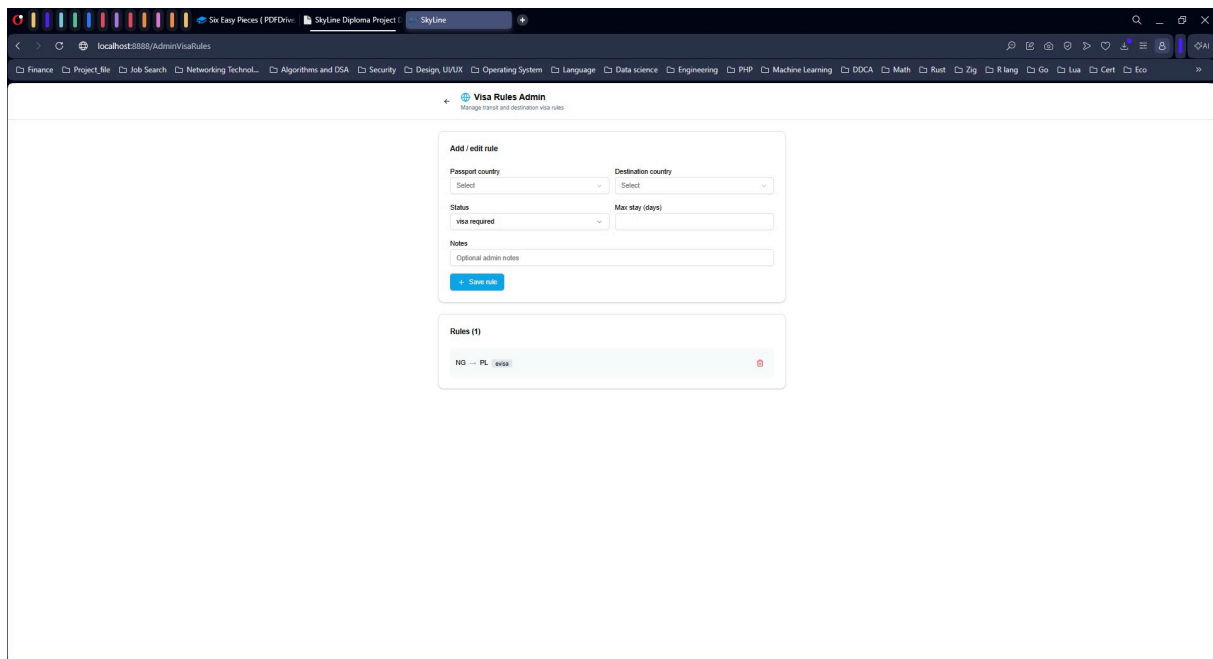
Country or region

Qatar

Continue



- Admin/rules screen for managing visa and transit rules in the diploma demo.



7. Code and Implementation Highlights

7.1. Netlify: Environment variables and API keys on netlify

```
//Environment variables
DUFFEL_ACCESS_TOKEN=*****
SUPABASE_URL=*****
SUPABASE_SERVICE_ROLE_KEY=*****
STRIPE_SECRET_KEY=*****
//used
#KIWI_TEQUILA_API_KEY=
#FLIGHTAWARE_API_KEY=
OPENSKY_USERNAME=*****
OPENSKY_PASSWORD=*****
VITE_SUPABASE_URL=*****
VITE_SUPABASE_ANON_KEY=*****
VITE_SUPABASE_PUBLISHABLE_KEY=*****
VITE_STRIPE_PUBLISHABLE_KEY=*****
```

7.2. Netlify Backend and lib API

```
├── /
│   └── netlify
│       └── functions
│           └── _lib
│               ├── duffelClient.js
│               ├── http.js
│               ├── kiwiClient.js
│               ├── openskyAuth.js
│               ├── routeScoring.js
│               ├── sampleAirports.js
│               └── supabaseAdmin.js
```

```

├── visaRules.js
├── aircraft-position.mjs
├── airports-search.mjs
├── book-flight.mjs
├── create-payment-intent.mjs
├── flights-cheapest-dates.mjs
├── flights-price.mjs
├── flights-search.mjs
├── flight-status.mjs
├── live-flights.mjs
├── visa-check.mjs

```

7.3. Frontend Architecture - /SkyLine/src/pages - /Skyline/src/ui

```

├── /
├── .netlify
├── dist
├── netlify
├── src
│   ├── assets
│   │   └── icon.svg
│   ├── components
│   │   ├── api
│   │   ├── travel
│   │   └── ui
│   │       └── //essential UI components
│   └── pages
│       ├── AcceptTerms.jsx
│       ├── AdminAirports.jsx
│       ├── AdminVisaRules.jsx
│       ├── AuthCallback.jsx
│       ├── Checkout.jsx
│       ├── EditTravelProfile.jsx
│       ├── HelpCenter.jsx
│       ├── Home.jsx
│       ├── LiveMap.jsx
│       ├── Login.jsx
│       ├── MyTrips.jsx
│       ├── PrivacyPolicy.jsx
│       ├── Profile.jsx
│       ├── Register.jsx
│       ├── RouteDetails.jsx
│       ├── SearchDetails.jsx
│       ├── Settings.jsx
│       └── TripDetails.jsx

```

7.4. Architecture Summary

Backend Functions	DB read	DB write	External API
airports-search	airports	-	-
flights-search	airports, visa_rules, transit_rules	-	Duffel
flights-cheapest-dates	-	-	kiwi-tequila
flight-status	-	-	FlightAware
aircraft-positions	-	-	OpenSky
live-flights	-	-	OpenSky
flights-price	-	-	Duffel
visa-check	visa-rules	-	-
create-payment-intent	-	-	Stripe
book-flight	-	trips	Stripe,Duffel

7.5. Sample Code - Frontend/Backend/DB Interaction

7.5.1. Airport search

- Frontend -> src/pages/Home + src/components/api/flightClient.mjs + src/components/travel/AirportSearchBox.jsx
- Backend -> .netlify/function/airports-search.mjs
- Database -> supabase/Airports

On the frontend, we check the data of our active profile and see if they are valid; validation is checked by passport country and visa strength. Frontend request used by forms and searchboxes to find airport using city, airport name or IATA/ICAO code. The main code that calls the backend is netlify/airports-search is:

```
const data = await getJSON('airports-search', { q: query });
return data.airports || [];
```

Full snippet:

```
//src/components/api/flightClients.js
// /**
//  * Airport search – used by FlightSearchForm, AirportSearchBox,
//  * EditProfileModal.
//  * Backed by the OurAirports dataset (imported into Supabase).
//  *
//  * @param {string} query free text – city, airport name, or IATA/ICAO code
//  * @returns {Promise<Airport[]>}
//  */
export async function searchAirports(query) {
  if (!query || query.length < 2) return [];
  try {
    const data = await getJSON('airports-search', { q: query });
    return data.airports || [];
  } catch {
    return [];
  }
}
```

```

//Frontend
//src/pages/Home
const runSearch = async (searchData) => {
  setLoading(true);
  try {
    let formattedTravelerProfile = null;
    // Wrap profile loading in its own safe try/catch block
    // so a database auth/fetch issue won't crash the flight search
    try {
      const profileData = await loadUserProfile();
      if (profileData && profileData.travel_profiles) {
        const { travel_profiles, active_profile_id } = profileData;
        const currentProfile = travel_profiles.find(p => String(p.id) ===
String(active_profile_id));

        if (currentProfile) {
          setActiveProfile(currentProfile);
          formattedTravelerProfile = {
            passport_country: currentProfile.passport_country,
            visas: currentProfile.visas || []
          };
        }
      }
    } catch (profileErr) {
      console.error("Failed to load user profile context:", profileErr);
      // Continues execution with formattedTravelerProfile = null
    }
  }
}

```

A proxy between the airport input, and database, then output the corresponding airport. A fail safe/fallback sample airport in case our database fails to respond

```

//Backend
//.netlify/function/airports-search.mjs

function toClientShape(row) {
  return {
    code: row.iata_code || row.icao_code || '',
    icao: row.icao_code,
    name: row.name,
    city: row.city,
    country: row.country,
    lat: row.latitude,
    lon: row.longitude,
    subType: row.type === 'large_airport' || row.type === 'medium_airport' ?
'AIRPORT' : 'CITY',
  };
}

try {
  const supabase = getSupabaseAdmin();

  const { data, error } = await supabase
    .from('airports')
    .select('iata_code, icao_code, name, city, country, latitude, longitude,
type')
    .or(

```

```

    [
      `iata_code.ilike.${query}%`,
      `icao_code.ilike.${query}%`,
      `city.ilike.${query}%`,
      `name.ilike.${query}%`,
    ].join(',')
  )
  .order('city', { ascending: true })
  .limit(15);

if (error) throw error;

const withIata = (data || []).filter((row) => row.iata_code);
const icaoOnly = (data || []).filter((row) => !row.iata_code);
const combined = [...withIata, ...icaoOnly];

if (combined.length > 0) {
  return ok({ airports: combined.map(toClientShape), source: 'supabase' });
}
//fail safe
const fallback = searchSampleAirports(query).map(toClientShape);
return ok({ airports: fallback, source: 'fallback' });
} catch (err) {
  console.error('airports-search error:', err);

  const fallback = searchSampleAirports(query).map(toClientShape);
  return ok({ airports: fallback, source: 'fallback', warning: err.message });
}
};

```

Database response:

```

{ "airports": [
  { "code": "DOH", "icao": "OTHH", "name": "Hamad International Airport",
    "city": "Doha", "country": "QA", "lat": 25.27, "lon": 51.61, "subType":
    "AIRPORT" }
  ], "source": "supabase" }
}]

```

Airport Dataset

id	iata_code	icao_code	name	city	country	latitude	longitude	type
2	NULL	OMAW	Abu Dhabi Northeast Airport - Suweih	Abu Dhabi	AE	24.526793	54.874812	small_airport
3	HIR	AGGH	Honara International Airport	Honara	SB	-9.428	90.054993	large_airport
4	MUA	AGOM	Munda Airport	Munda	SB	-8.32797	157.263	medium_airport
5	NULL	NULL	Gjrokastër Airfield	Gjrokastër	AL	40.087301	20.152302	closed
7	NULL	NULL	Arzhi Airport	Nar Hachn	AM	40.294302	44.594802	small_airport
8	NUJ	ANYN	Nazru International Airport	Yaren	NR	-5.547893	166.819525	medium_airport
9	NULL	NULL	Matals Airport	Matals	AO	-14.727499991853	15.01399999380649	small_airport
10	NULL	NULL	Cabo Ledo Air Base	Cabo Ledo	AO	-9.85305	13.2806	small_airport
11	NULL	NULL	Palmer Station Airport	Palmer Station	AQ	-64.774582	-64.039907	small_airport
12	NULL	NULL	Petrel Base	Petrel Base	AQ	-63.479	-66.2333	small_airport
13	NULL	NULL	Cachi Airport	Cachi	AR	-25.1049995432	-66.1572036743	small_airport
14	NULL	NULL	Cafayate Airport	Cafayate	AR	-26.056301	-65.936897	small_airport
15	NULL	NULL	Villa Minetti Airport	Villa Minetti	AR	-18.6201	-61.6038	small_airport
16	NULL	SAAK	Isla Martín García Airport	Isla Martín García	AR	-34.1821	-58.2489	small_airport
17	NULL	NULL	Bell Ville Airport	Bell Ville	AR	-32.6599	-62.702	small_airport
18	NULL	NULL	Santa Rita Airport	Saladillo	AR	-35.8283	-60.1467	small_airport
19	NULL	NULL	El Porton Airport	Buta Ranquil	AR	-37.8432	-69.612619	small_airport
20	NULL	NULL	Aeródromo Estancia Don Panos	Presidencia Roca	AR	-26.2959	-59.5353	small_airport
21	NULL	NULL	Campo Arenal Airport	Campo Arenal	AR	-27.0723	-66.550088	small_airport
22	NULL	NULL	Quemu Quemu Airport	Quemu Quemu	AR	-36.0587	-63.6332	small_airport

7.5.2. Search flights + Flight pricing + Create Payment Intent

- Frontend -> src/pages/Home.js + src/components/api/flightClient.js
- Backend -> netlify/flights-search.mjs + netlify/flights-price.mjs + netlify/lib/duffelClient.js

- Database -> Airports, visa_rules, transit_rules

The part that calls the netlify backend flights-search:

```
const data = await postJSON('flights-search', {
  origin: originCode,
  destination: destinationCode,
  departureDate,
  returnDate: returnDate || null,
  travelerProfile: travelerProfile || null,
});
return data.offers || [];
```

Full snippet:

```
//src/components/api/flightClient.js
/**
 * Flight search – used by Home.jsx after FlightSearchForm submits.
 * Routed through Duffel, annotated with visa risk, and ranked using the
 * documented price/duration/comfort/visa weighting model.
 *
 * @param {FlightSearchParams} [params]
 * @returns {Promise<Array<Object>>} ranked route offers
 */
export async function searchFlights({ origin, destination, departureDate,
returnDate, travelerProfile } = {}) {
  const originCode = typeof origin === 'string' ? origin : origin?.code;
  const destinationCode = typeof destination === 'string' ? destination :
destination?.code;

  try {
    const data = await postJSON('flights-search', {
      origin: originCode,
      destination: destinationCode,
      departureDate,
      returnDate: returnDate || null,
      travelerProfile: travelerProfile || null,
    });
    return data.offers || [];
  } catch (err) {
    if (import.meta.env.DEV) {
      // @ts-ignore
      console.warn('searchFlights falling back to mock data:', err.message);
      return generateMockOffers(
        {
          origin: typeof origin === 'string' ? { code: origin } : origin,
          destination: typeof destination === 'string' ? { code: destination } :
destination,
          departureDate,
        },
        travelerProfile
      );
    }
    throw err;
  }
}
```

```

//searching frontend
//Pass payload safely into searchFlights
const offers = await searchFlights({
  origin: searchData.origin,
  destination: searchData.destination,
  departureDate: searchData.departureDate,
  returnDate: searchData.returnDate,
  travelerProfile: formattedTravelerProfile,
});

setRoutes(offers || []);
saveSearchResults(offers || []);

const fromDate = formatDate(addDays(new Date(searchData.departureDate),
-15), 'yyyy-MM-dd');
const toDate = formatDate(addDays(new Date(searchData.departureDate), 15),
'yyyy-MM-dd');
const prices = await getCheapestDates({
  origin: searchData.origin.code,
  destination: searchData.destination.code,
  fromDate,
  toDate,
});
setCalendarPrices(Object.keys(prices).length ? prices : null);
} catch (err) {
  console.error("Flight search endpoint error:", err);
  toast.error(err.message || 'Search failed');
  setRoutes([]);
} finally {
  setLoading(false);
}
};

const handleSelectRoute = (route) => {
  saveSelectedRoute(route);
  navigate(createPageUrl('RouteDetails'));
};

const handleCalendarDateSelect = async (date) => {
  const next = { ...search, departureDate: formatDate(date, 'yyyy-MM-dd') };
  await runSearch(next);
  setViewMode('list');
};

if (!search) return null;

const filtered = filterRoutes(routes, filter);

```

Database response is to read the *airports*, *visa_rules*, and *transit_rules* tables via *visaRules.js* then annotate each Duffel offer, returning json response:

```

offers": [
  { "id": "off_123", "price": 300, "hasVisaIssue": false, "isRecommended":
true, "score": 92, "destVisaStatus": "visa_free" }
] }
]

```

User flow moves towards showing a list of flights, prices, and offers: Frontend call *priceOffer.js*:


```

/**
 * Price a previously-searched offer and lock it for booking.
 * Maps to Duffel's offer-price step before order creation.
 *
 * @param {string} offerId
 * @returns {Promise<Object>}
 */
export async function priceOffer(offerId) {
  try {
    return await postJSON('flights-price', { offerId });
  } catch (err) {
    if (import.meta.env.DEV) {
      const route = JSON.parse(sessionStorage.getItem('skyline_selected_route')
|| 'null');
      return {
        offerId,
        totalAmount: route?.price || 450,
        currency: route?.currency || 'USD',
        available: true,
      };
    }
    throw err;
  }
}

```

Order and response Information in Duffel:

Orders

New order

All Needs review On hold Pending

+ Filter

Export

AIRLINE	BOOKING REF	STATUS	ID	FAMILY NAMES	NEXT DEPARTURE	AMOUNT	CREATION DATE ↓
 ZZ	FBKDU4	Past	ord_0000B8mCdoFDrVCn19dIXR	doee	—	GBP 300.31	7/28/2026
 BA	2DWH30	Past	ord_0000B8jYkq3ZRHpr2SlnZt	doee	—	GBP 166.78	7/26/2026
 ZZ	EHVQUJ	Past	ord_0000B8dLPnZJSduKW1BQae	doee	—	GBP 161.94	7/23/2026
 BA	2H2QAP	Past	ord_0000B8dL10P1KcGnYG4w9M	doee	—	GBP 169.10	7/23/2026
 ZZ	PYRBF8	Past	ord_0000B8YcfcgBJoMyYtnG9z1	Doe	—	GBP 170.65	7/21/2026
 ZZ	OVIHJA	Past	ord_0000B8YSdCRhfs91nQqn04	Doe	—	GBP 169.65	7/21/2026

7.5.3. Create Payment Intent

- Frontend -> src/pages/Checkout.jsx + src/component/api/flightClient.js
- Backend -> netlify/functions/create-payment-intent.mjs + stripe API
- Database -> No write/read

```

src/components/api/flightClient.js
export async function createPaymentIntent({ offerId, amount, currency }) {
  try {
    return await postJSON('create-payment-intent', { offerId, amount,
currency });
  } catch (err) {
    if (import.meta.env.DEV) {
      throw new Error('Stripe not available in dev without Netlify – run `netlify
dev` or use demo checkout.');
```

```

    }
    throw err;
  }
}

```

handling payment intent:

```

netlify/functions/create-payment-intent.mjs
const { offerId, amount, currency } = parseBody(event);
if (!offerId || !amount || !currency) {
  return badRequest('offerId, amount, and currency are required.');
```

```

}

try {
  const stripe = getStripe();
  const paymentIntent = await stripe.paymentIntents.create({
    amount: Math.round(amount),
    currency: currency.toLowerCase(),
    metadata: { offerId },
    automatic_payment_methods: { enabled: true },
  });
}

```

Stripe Payment Intent and response:

```

{ "clientSecret": "pi_123_secret_abc", "paymentIntentId": "pi_123" }

```

```

Request from ::1: POST /.netlify/functions/flights-price
Response with status 200 in 338 ms.
Request from ::1: POST /.netlify/functions/create-payment-intent
Response with status 200 in 1002 ms.

```

7.5.4. Book Flight

- Frontend -> src/pages/TripsDetails.jsx + src/pages/MyTrips.jsx
- Backend/Router -> netlify/functions/book-flight.mjs + src/components/api/flightClient.js
- Database -> Insert into *trips* table

Snippet that bridges/communicates with the *book-flight* backend:

```

//src/components/api/flightClient.js

/**
 * Create a booking/order. Maps to Duffel order creation. Should be called
 * only after a successful Stripe PaymentIntent confirmation.
 */
* @param {Object} params
* @param {string} params.offerId
* @param {Array<Object>} params.passengers
* @param {string} params.paymentIntentId
* @param {string} [params.userId]
* @returns {Promise<Object>} booking record (also persisted to Supabase)
*/
export async function createBooking({ offerId, passengers, paymentIntentId,
userId }) {
  try {

```

```

    return await postJSON('book-flight', { offerId, passengers, paymentIntentId,
    userId });
  } catch (err) {
    if (import.meta.env.DEV) {
      return { bookingReference: `SKY-${Date.now().toString(36).toUpperCase()}`,
    orderId: offerId };
    }
    throw err;
  }
}

```

Backend that communicates with stripe, duffel, and then plot the information into our *trips* table:

- Confirmation with stripe:

```

//netlify/functions/book-flight.mjs

function getStripe() {
  const key = process.env.STRIPE_SECRET_KEY;
  if (!key) throw new Error('STRIPE_SECRET_KEY is not configured.');
```

```

  return new Stripe(key);
}

// 1. Confirm payment actually succeeded before issuing the ticket.
const stripe = getStripe();
const paymentIntent = await stripe.paymentIntents.retrieve(paymentIntentId);
if (paymentIntent.status !== 'succeeded') {
  return badRequest(`Payment has not completed (status:
${paymentIntent.status}).`);
}

```

- Use our passenger information and create an order on duffel - which will help with printing out the iternary

information.

```

const orderResult = await createOrder({ offerId, passengers:
formattedPassengers });
const order = orderResult?.data;

if (!order) {
  return serverError('Failed to retrieve order data from Duffel.');
```

```

}

```

- Retrieve the information from duffel(flight information, passenger information, etc.) and place them into

our database - which will help when placing information in our frontend(MyTrips, TripDetails).

```

const supabase = getSupabaseAdmin();
const tripRecord = {
  booking_reference: order.booking_reference,
  duffel_order_id: order.id,
  user_id: userId || null,
  status: 'upcoming',

```

```

        origin: order.slices[0]?.segments[0]?.origin,
        destination: order.slices[order.slices.length - 1]?.segments?.slice(-1)
[0]?.destination,
        departure_date: order.slices[0]?.segments[0]?.departing_at,
        arrival_date: order.slices[order.slices.length - 1]?.segments?.slice(-1)
[0]?.arriving_at,
        price: parseFloat(order.total_amount),
        currency: order.total_currency,
        payment_intent_id: paymentIntentId,
        passengers: formattedPassengers,
        segments: order.slices.flatMap((slice) => slice.segments),
        created_at: new Date().toISOString(),
    };

    const { data: savedTrip, error: insertError } = await supabase
        .from('trips')
        .insert(tripRecord)
        .select()
        .single();

    if (insertError) {
        // Booking with the airline already succeeded – don't fail the
        // whole request just because the local record failed to save.
        console.error('Failed to persist booking to Supabase:', insertError);
    }

    return ok({
        bookingReference: order.booking_reference,
        orderId: order.id,
        trip: savedTrip || tripRecord,
    });
} catch (err) { //book flight fail
    console.error('book-flight error:', err);
    return serverError(err.message || 'Booking failed.');
```

Database response: verify Stripe paymentintent, call Duffels order-create endpoint, then INSERT into the *trips* table:

- Exemplary response from duffel:

```
{ "bookingReference": "FBKDU4", "orderId": "ord_456", "trip": { "id": 17,
"user_id": "uuid-...", "price": 300, "currency": "USD" } }
```

- Skyline *trips* database:

```
{ "idx": 0, "id": 1, "user_id": "d6230797-
e889-42a1-9dfb-14a4b67f10ca", "booking_reference": "0VIHJA", \
"duffel_order_id": "ord_0000B8YSdCRhfs9lnQn04", "status": "upcoming", "origin": " \
{ \"iata_city_code\": \"DOH\", \
\"city_name\": \"Doha\", \"iata_country_code\": \"QA\", \"icao_code\": \"OTH\", \
\"iata_code\": \"DOH\", \
\"latitude\": 25.260642, \"longitude\": 51.615118, \"city\": null, \"time_zone\": \
\"Asia/Qatar\", \
\"type\": \"airport\", \"name\": \"Hamad International Airport\", \"id\": \
\"arp_doh_qa\" }\", \
\"destination\": { \"iata_city_code\": \"WAW\", \"city_name\": \"Warsaw\",
```

```

\ "iata_country_code\": \
\ "PL\","icao_code\":"EPWA\","iata_code\":"WAW\","latitude\":"52.166211,
\ "longitude\":"20.967363, \
\ "city\":"null\","time_zone\":"Europe/Warsaw\","type\":"airport\","name\":"
\ "Warsaw Chopin Airport\"," \
\ "id\":"arp_waw_pl\","}","departure_date":"2026-07-29
21:40:00+00","arrival_date":"2026-07-30 02:30:00+00", \
\ "price":"169.65","currency":"GBP","payment_intent_id":"pi_3TvYid6ISHie8VVgldGkbbk8h",
\

```

Results on duffel:

AIRLINE	BOOKING REF	STATUS	ID	FAMILY NAMES	NEXT DEPARTURE	AMOUNT	CREATION DATE ↓
 ZZ	WC62MN	Confirmed	ord_0000BA4uDA47ZWushP7QsC	Doe	9/23/2026 5:01 PM	GBP 161.31	9/5/2026
 ZZ	CAUAIO	Confirmed	ord_0000BA2JmA4oboPrbcKedU	Doe	9/16/2026 11:36 AM	GBP 165.29	9/3/2026
 ZZ	FBKDU4	Past	ord_0000B8mCdoFDrVCn19dIXR	doee	—	GBP 300.31	7/28/2026
 BA	2DWH3O	Past	ord_0000B8jYkq3ZRHpr2S1nZt	doee	—	GBP 166.78	7/26/2026
 ZZ	EHVQUJ	Past	ord_0000B8dLPnZJSduKW1BQae	doee	—	GBP 161.94	7/23/2026
 BA	2H2QAP	Past	ord_0000B8dL10P1KcGnYG4w9M	doee	—	GBP 169.10	7/23/2026
 ZZ	PYRBFB	Past	ord_0000B8YcFgBJoWyYtnG9z1	Doe	—	GBP 170.65	7/21/2026
 ZZ	OVIHJA	Past	ord_0000B8YSdCRhfs91nQQn04	Doe	—	GBP 169.65	7/21/2026

8. Conclusions and Development Prospects

SkyLine addresses a practical problem that is not fully solved by ordinary flight search engines. Many travelers cannot choose flights based only on price because their passport strength, residence status, and visas affect which destinations and transit routes are possible. The project demonstrates how flight APIs, airport data, user profiles, and visa/transit rules are be combined into a recommendation system that provides safer and more useful travel planning

The current version uses a controlled visa rule dataset and API sandbox environments. Future development could include integration with IATA Timatic or Sherpa for professional travel documentation checks, wider booking inventory through Sabre or Travelport, automatic passport document scanning, multilingual support, mobile applications, route risk learning based on user feedback, and stronger admin tools for maintaining visa rules.

Github: <https://github.com/Otu-EkongEffiong/SkyLine>

Website: gilded-rugelach-aa6f62.netlify.app

9. Bibliography and Sources

- Duffel API Documentation: <https://duffel.com/docs/api/overview/welcome> (Accessed: 14.07.2026)
- Kiwi.com Tequila API Documentation: https://tequila.kiwi.com/portal/docs/tequila_api (Accessed: 22.06.2026)
- FlightAware AeroAPI Documentation: <https://www.flightaware.com/aeroapi/portal/documentation> (Accessed: 05.08.2026)
- OpenSky Network REST API: <https://openskynetwork.github.io/opensky-api/rest.html> (Accessed: 11.07.2026)
- OurAirports Data: <https://ourairports.com/data/> (Accessed: 29.08.2026)
- Supabase Documentation: <https://supabase.com/docs> (Accessed: 19.06.2026)

- Stripe API Documentation: <https://docs.stripe.com/api> (Accessed: 02.09.2026)
- Netlify Functions Documentation: <https://docs.netlify.com/build/functions/overview/> (Accessed: 15.08.2026)
- IATA Timatic: <https://www.iata.org/en/services/compliance/timatic/> (Accessed: 08.06.2026)
- Sherpa: <https://www.joinsherpa.com/> (Accessed: 27.07.2026)
- Passport Index: <https://www.passportindex.org/> (Accessed: 04.09.2026)
- Henley Passport Index: <https://www.henleyglobal.com/passport-index> (Accessed: 13.08.2026)